

# Carte de Referinta

Tehnici Machine Learning Moderne

Manual complet pentru pipeline-ul ML: de la data cleaning la deployment.

12 categorii x 150+ tehnici - cu logica matematica, exemple si scenarii de utilizare

Diana Nenu | Lucrare Disertatie | Master Data Science and AI  
Bucuresti, 2026

# CUPRINS

|                                                    |    |
|----------------------------------------------------|----|
| Introducere .....                                  | 3  |
| Categoria 1. Data Cleaning .....                   | 4  |
| Categoria 2. Exploratory Data Analysis (EDA) ..... | 8  |
| Categoria 3. Feature Engineering .....             | 11 |
| Categoria 4. Feature Selection .....               | 16 |
| Categoria 5. Data Splitting si Validation .....    | 19 |
| Categoria 6. Modele de Machine Learning .....      | 22 |
| Categoria 7. Hyperparameter Tuning .....           | 29 |
| Categoria 8. Metrice de Evaluare .....             | 31 |
| Categoria 9. Explainability (XAI) .....            | 34 |
| Categoria 10. Deployment .....                     | 37 |
| Categoria 11. Monitoring .....                     | 39 |
| Categoria 12. Tehnici Emergente (2024-2025) .....  | 41 |
| Cum sa folosesti aceasta carte .....               | 44 |

# Introducere

Acest document este un manual de referinta pentru tehnicile moderne de Machine Learning. Pentru fiecare tehnica este descris: cum functioneaza (logica matematica si informatica), cand se foloseste, cand NU se foloseste, limitarile, frecventa de utilizare in 2025 si tehnicile complementare.

Documentul este structurat pe 12 categorii mari care acopera intregul pipeline ML, de la colectarea datelor pana la deployment si monitoring. Fiecare categorie contine multiple tehnici, ordonate de la fundamentale la avansate.

Acesta e gandit ca o referinta - poate fi citit linear sau consultat punctual. La fiecare tehnica gasesti exemple concrete de cod si scenarii de utilizare.

# Categoria 1. Data Cleaning

Curatarea datelor este etapa unde corectam problemele cunoscute: valori lipsa, duplicate, anomalii, inconsistente, tipuri gresite. Aceasta este separata de EDA - aici modificam efectiv datele.

## 1.1. Tratarea valorilor lipsa

### A. dropna() - eliminarea randurilor cu NaN

Logica: cea mai simpla solutie - daca un rand are valori lipsa, il elimini complet.

```
df_clean = df.dropna() # elimina daca ORICE coloana e NaN
df_clean = df.dropna(subset=['col1', 'col2']) # doar daca specifice coloane sunt NaN
df_clean = df.dropna(thresh=5) # tine doar randurile cu cel putin 5 valori non-NaN
```

Cand se foloseste: cand NaN-urile sunt rare (< 5% din date) si distribuite aleator. Cand pierderea de date nu afecteaza semnificativ analiza.

Cand NU: cand NaN-urile sunt frecvente (>10%). Daca elimini 30% din date, modelul antrenat nu mai e reprezentativ.

Limitari: pierdere de informatie. Daca NaN-urile nu sunt aleatoare, introduci bias.

Frecventa: foarte des. Aproape orice pipeline are macar un dropna() la final.

Combina cu: e ultima linie de aparare dupa imputare. Eu am facut interpolare apoi dropna() pentru cele care nu s-au putut interpola.

### B. Imputare cu mean / median / mode

Logica: completezi NaN cu o valoare reprezentativa a coloanei.

```
df['col'].fillna(df['col'].mean()) # numerice cu distributie aproape normala
df['col'].fillna(df['col'].median()) # numerice cu outliers (mai robust)
df['col'].fillna(df['col'].mode()[0]) # categorice
```

Cand se foloseste: feature-uri ne-temporale, cand NaN-urile sunt putine si aleatoare.

Cand NU: pentru time series (introduci o discontinuitate). Cand distributia coloanei e bimodala.

Limitari: reduce varianta naturala a datelor. Modelul invata o concentratie artificiala la valoarea de imputare.

Frecventa: foarte des in date tabulare clasice.

### C. Forward Fill / Backward Fill

Logica: pentru NaN, folosesti ultima valoare cunoscuta (forward) sau prima valoare urmatoare (backward).

```
df['col'].fillna(method='ffill') # ultima valoare cunoscuta
df['col'].fillna(method='bfill') # urmatoarea valoare cunoscuta
```

Cand se foloseste: time series cu schimbari lente. Senzori care raporteaza periodic.

Cand NU: cand seria are schimbari bruste.

Frecventa: standard pentru time series scurte.

### D. Interpolare liniara

Logica: trasezi o linie dreapta intre punctele cunoscute si iei valorile pe ea. Matematic: pentru NaN intre  $x_a$  si  $x_b$  la timpul  $t$ :  $\text{NaN\_value} = x_a + (x_b - x_a) * (t - t_a) / (t_b - t_a)$

```
df['col'].interpolate(method='linear', limit=24)
df['col'].interpolate(method='cubic')
df['col'].interpolate(method='time')
```

Cand se foloseste: time series cu schimbari graduale. Date senzoriale.

Limitari: pe goluri lungi presupune o continuitate care poate sa nu existe. Solutie: limita limit=N.

Frecventa: standard pentru time series ne-volatile.

## E. KNN Imputer

Logica: pentru fiecare valoare lipsa, foloseste cele mai apropiate K observatii bazate pe celelalte features. Calculezi media valorilor lor.

Algoritm: 1) Pentru un rand cu NaN in coloana C; 2) Calculezi distanta euclidiana cu toate celelalte randuri folosind doar coloanele NON-NaN; 3) lei cei mai apropiati K vecini; 4) Imputezi cu media valorilor lor in coloana C.

```
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors=5, weights='distance')
df_imputed = imputer.fit_transform(df)
```

Cand: date tabulare cu features corelate.

Limitari: scump computational. Sensibil la scala.

Frecventa: foarte folosit in dataset-uri tabulare clasice.

## F. MICE (Multiple Imputation by Chained Equations)

Logica: model iterativ. Pentru fiecare coloana cu NaN, antreneaza un model de regresie folosind celelalte coloane ca features. Repeta pana la convergenta.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
imputer = IterativeImputer(max_iter=10, random_state=42)
df_imputed = imputer.fit_transform(df)
```

Cand: cercetare medicala, sociologie.

Frecventa: standard de aur in academia, mai rar in industrie.

## 1.2. Tratarea outliers

### A. IQR (Interquartile Range)

Logica: outliers = valori in afara intervalului  $[Q1 - k \cdot IQR, Q3 + k \cdot IQR]$  unde  $IQR = Q3 - Q1$ . Standardul foloseste  $k = 1.5$  (sensibil) sau  $k = 3$  (conservator).

Matematic: bazat pe quartile ( $Q1 = \text{percentila } 25$ ,  $Q3 = \text{percentila } 75$ ). Robust la outliers (deoarece quartilele nu se schimba dramatic).

```
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
mask = (df['col'] < Q1 - 1.5*IQR) | (df['col'] > Q3 + 1.5*IQR)
df_clean = df[~mask]
```

Cand: distributii cu coada lunga (preturi, salarii).

Frecventa: foarte des, prima alegere.

## B. Z-score

Logica: outliers = valori cu z-score absolut > prag (tipic 3). Matematic:  $z = (x - \text{mean}) / \text{std}$ .

Cand: distributii aproape normale. Cand NU: distributii skewed.

## C. Isolation Forest

Logica: outliers sunt mai usor de izolat in arbori aleatori. Daca pentru o observatie ai nevoie de putine taieturi pentru a o izola, e probabil outlier.

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.05, random_state=42)
outlier_labels = iso.fit_predict(df) # -1 = outlier, 1 = normal
```

Cand: detectie anomalii in date multidimensionale (fraud detection).

Frecventa: foarte popular. 2025: standard pentru fraud detection in fintech.

## D. DBSCAN si LOF

DBSCAN: clustering bazat pe densitate. Punctele in zone cu putine puncte vecine sunt marcate ca noise.

LOF: pentru fiecare punct, compara densitatea sa locala cu cea a vecinilor.

## Categoria 2. Exploratory Data Analysis (EDA)

EDA este etapa unde te uiti la date sa le intelegi - inainte sa le transformi. Aici nu modifici datele, doar le observi.

### 2.1. Statistici descriptive

`describe()` - rezumat numeric: count, mean, std, min, 25%, 50%, 75%, max. Standard absolut, prima actiune dupa incarcarea datelor.

### 2.2. Vizualizari

Histograme si density plots: vizualizeaza distributia unei variabile numerice. Standard pentru analiza univariata.

Box plots: bazat pe quartile. Arata mediana, IQR, outliers. Bun pentru comparatii intre grupuri.

Scatter plots: relatia intre doua variabile. Variante moderne: hexbin (milioane puncte), joint plot (scatter + histograme marginale).

### 2.3. Corelatii

#### Pearson

Logica: corelatie liniara intre doua variabile. Matematic:  $r = \text{cov}(X, Y) / (\text{std}(X) * \text{std}(Y))$ . Valoare in  $[-1, 1]$ .

Cand: relatii liniare. Cand NU: relatii neliniare. Limitari: sensibil la outliers. Frecventa: 95%, default.

#### Spearman

Logica: corelatia rangurilor. Capteaza relatii monotone (oricat de neliniare). Cand: relatii monotone non-liniare, outliers prezenti.

#### Kendall

Bazat pe perechi concordante/discordante. Mai conservator. Folosit doar in studii statistice riguroase.

### 2.4. Mutual Information

Logica: cat de mult informatia despre o variabila reduce incertitudinea despre alta. Capteaza si relatii neliniare. Matematic:  $MI(X, Y) = H(Y) - H(Y|X)$  unde H e entropia.

```
from sklearn.feature_selection import mutual_info_regression
mi_scores = mutual_info_regression(X, y)
```

Cand: relatii complexe neliniare. Frecventa: standard modern pentru feature selection.

### 2.5. Profiling automat

ydata-profiling: genereaza un raport HTML complet automat cu toate statisticile, distributiile, corelatiile, valorile lipsa, alerte.

```
from ydata_profiling import ProfileReport
profile = ProfileReport(df, title='EDA Report')
profile.to_file('eda_report.html')
```

Frecventa: in crestere puternica. Multi data scientisti incep cu asta.

## 2.6. Hypothesis testing

Chi-square test: independenta intre doua variabile categorice. T-test: compararea mediilor a doua grupuri.

Mann-Whitney U: alternativa non-parametrica la t-test. Kolmogorov-Smirnov: testarea a doua distributii.

Cand: pentru a raspunde la intrebari ca diferenta dintre grupul A si B e statistically significant?



## Categoria 3. Feature Engineering

Transformarea datelor brute in features informative pentru model. Aceasta etapa face diferenta intre un model mediocru si unul exceptional. Cunoasterea domeniului conteaza enorm aici.

### 3.1. Encoding pentru variabile categorice

#### Label Encoding

Logica: mapeaza fiecare categorie la un numar (0, 1, 2). Cand: variabile ordinale (small < medium < large), targeturi pentru clasificare. Cand NU: variabile nominale (rosu, verde) - introduce ordine artificiala.

#### One-Hot Encoding

Logica: o coloana per categorie, valori binare 0/1. Dimensionalitate creste cu N (numar categorii).

```
pd.get_dummies(df['col'])
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(sparse_output=False, drop='first')
```

Cand: variabile nominale cu cardinalitate mica (< 20 categorii). Cand NU: 1000+ categorii (explozie dimensionalitate). Frecventa: 90%, default pentru categorice nominale.

#### Target Encoding

Logica: inlocuiesti fiecare categorie cu media target-ului pentru acea categorie.

```
from category_encoders import TargetEncoder
te = TargetEncoder(smoothing=10)
X_encoded = te.fit_transform(X, y)
```

Cand: cardinalitate inalta (1000+). Limitari: leakage potential - se rezolva cu smoothing. Frecventa: foarte popular in competitii Kaggle.

#### Frequency Encoding, Binary Encoding, Hashing

Frequency Encoding: inlocuieste cu frecventa categoriei. Binary Encoding: combinatie hash + binary, util pentru categorice cu multe valori. Hashing Trick: hash determinist - util cand ai milioane categorii.

### 3.2. Encoding pentru variabile temporale

#### Datetime decomposition

```
df['hour'] = df.index.hour
df['dayofweek'] = df.index.dayofweek
df['month'] = df.index.month
df['year'] = df.index.year
df['quarter'] = df.index.quarter
df['is_weekend'] = (df.index.dayofweek >= 5).astype(int)
```

Frecventa: 100% in time series.

#### Encoding ciclic (sin/cos)

Logica: pentru variabile cu modulo natural (24, 7, 12). Plasezi orele pe un cerc unitate astfel ca vecinatatea matematica sa corespunda cu cea reala.

```
df['hour_sin'] = np.sin(2 * np.pi * df['hour'] / 24)
df['hour_cos'] = np.cos(2 * np.pi * df['hour'] / 24)
```

Cand: ore, zile saptamana, luni, unghiuri busola, faze luna. Cand NU: variabile liniare (an, varsta).

### Holiday flags

```
import holidays
us_holidays = holidays.US()
df['is_holiday'] = df.index.normalize().isin(us_holidays.keys()).astype(int)
```

Frecventa: standard in retail, energy, traffic.

### Fourier features

Logica: descompui seria in componente sinusoidale de frecvente diferite. Util pentru sezonalitati complexe care nu se incadreaza in cicluri standard.

Frecventa: folosit intern de Prophet. Util pentru retele neurale custom.

## 3.3. Scalare si transformari

### StandardScaler

Logica:  $(x - \text{mean}) / \text{std} \rightarrow$  medie 0, std 1.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Cand: distributii aproape normale, algoritmi liniari. Cand NU: pe arbori. Limitari: sensibil la outliers. Frecventa: 90% pentru DL si modele liniare.

### MinMaxScaler

Logica:  $(x - \text{min}) / (\text{max} - \text{min}) \rightarrow$  interval [0, 1]. Cand: retele neurale cu activari sigmoid/tanh. Cand NU: cu outliers.

### RobustScaler

Logica:  $(x - \text{median}) / \text{IQR}$ . Robust la outliers. Cand: distributii cu outliers reali.

### QuantileTransformer si PowerTransformer

QuantileTransformer: mapeaza distributia la una uniforma sau normala folosind quantilele. PowerTransformer (Box-Cox, Yeo-Johnson): face datele aproape gaussiene.

```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method='yeo-johnson')
```

### Log transformation

Logica:  $\log(1 + x)$ . Pentru distributii cu coada lunga (preturi, venituri). Frecventa: foarte des, default pentru values that span orders of magnitude.

## 3.4. Time Series Feature Engineering

### Lag features

Logica: `target.shift(N)`. Pentru fiecare moment, valoarea target-ului din N pasi in urma. Cei mai puternici predictorii in serii cu sezonalitati ciclice.

## Rolling features

Logica: `target.shift(1).rolling(N).mean()`. Statistica (mean, std, min, max) pe o fereastră mobilă. CRITICAL: aplica `shift(1)` înainte de rolling pentru a evita data leakage.

## Expanding features

Logica: medii cumulative de la începutul seriei până la momentul curent.

```
df['target_cumulative_mean'] = df['target'].expanding().mean()
```

## Differencing

Logica:  $y_{\text{diff}}(t) = y(t) - y(t-1)$ . Elimina trendul. Cand: pentru ARIMA. Frecvență: standard pentru ARIMA, mai rar în ML modern.

## Rate of change

```
df['target_pct_change'] = df['target'].pct_change()
```

Cand: în finance pentru returns. În retail pentru creșteri procentuale.

## 3.5. Domain-specific features

Exemple: Finance: Sharpe Ratio, RSI, MACD, Bollinger Bands. Solar: Performance Ratio, `dc_ac_ratio`, temperature derating. Retail: Customer Lifetime Value, RFM. Medical: BMI, MELD score.

Cum se construiesc: cunoaște domeniul + înțelege ce întrebări de business sunt relevante. Asta diferențează un Data Scientist senior de un junior.

## 3.6. Polynomial features și crosses

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, interaction_only=False)
X_poly = poly.fit_transform(X)
```

Cand: relații neliniare în modele liniare. Limitări: explozie dimensionalitate ( $N \rightarrow N^2$ ).

## 3.7. Featuretools și TSFresh

Featuretools: generează automat sute de features din relații între tabele (Deep Feature Synthesis).

TSFresh: extrage 800+ features automate pentru time series (entropie, complexitate, frecvențe Fourier, autocorelații).

## Categoria 4. Feature Selection

Eliminarea features-urilor redundante sau zgomotoase pentru a reduce dimensionalitatea si a imbunatati generalizarea.

### 4.1. Variance Threshold

```
from sklearn.feature_selection import VarianceThreshold
vt = VarianceThreshold(threshold=0.01)
X_filtered = vt.fit_transform(X)
```

Frecventa: simplu si util. Mereu primul pas.

### 4.2. Correlation removal

Din doua features puternic corelate (Pearson > 0.95), pastrezi una. Cand: features generate automat (TSFresh, Featuretools).

### 4.3. Recursive Feature Elimination (RFE)

```
from sklearn.feature_selection import RFE
rfe = RFE(estimator=RandomForestRegressor(), n_features_to_select=20)
X_reduced = rfe.fit_transform(X, y)
```

Cand: vrei un numar fix de features. Limitari: lent (de N ori mai lent decat antrenarea simpla).

### 4.4. Tree-based Feature Importance

```
model = RandomForestRegressor()
model.fit(X, y)
importances = pd.Series(model.feature_importances_, index=X.columns)
```

Avantaj: rapid (output direct al antrenarii). Limitari: bias spre features cu cardinalitate mare. Frecventa: 100% in proiecte cu RF/XGBoost.

### 4.5. Permutation Importance

Logica: pentru fiecare feature, shuffle valorile si masoara cat scade performanta. Drop mare = important.

```
from sklearn.inspection import permutation_importance
perm = permutation_importance(model, X, y, n_repeats=10)
```

Avantaj: model-agnostic, robust. Frecventa: in crestere - mai bun decat feature\_importances\_ standard.

### 4.6. SHAP-based Selection

Calculezi SHAP values, alegi top-K features cu valoare medie absoluta cea mai mare. Standardul modern pentru selectie in proiectele profesionale.

### 4.7. Boruta / BorutaShap

Logica: testeaza fiecare feature impotriva unor shadow features (copii shuffled). Pastreaza features clar mai bune decat random.

```
from boruta import BorutaPy
```

```
boruta = BorutaPy(RandomForestRegressor(), max_iter=100)
boruta.fit(X.values, y.values)
```

Avantaj: rigorous statistic. Frecventa: foarte popular in competitii Kaggle si in pharma.

## 4.8. Lasso (L1 regularization)

Logica: regresie liniara cu penalitate  $\lambda * \sum(|coef|)$ . Forteaza coeficientii inutili la zero. Matematic: minimizeaza  $MSE + \lambda * L1(coef)$ .

```
from sklearn.linear_model import LassoCV
lasso = LassoCV(cv=5)
lasso.fit(X, y)
non_zero_features = X.columns[lasso.coef_ != 0]
```

Cand: cand suspectezi multe features inutile. Limitari: alege arbitrar dintre features corelate. Frecventa: 80% in proiecte cu modele liniare.

## Categoria 5. Data Splitting si Validation

Sa-ti evaluezi modelul pe date pe care nu le-a vazut, simuland conditii reale de productie. Aici anti-leakage devine esential.

### 5.1. Random train/test split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Cand: date independente, fara structura temporala. Cand NU: time series (introduce data leakage).  
Frecventa: 80% pentru ML traditional.

### 5.2. Stratified Split

Pastreaza proportia claselor target in ambele seturi. Cand: clasificare cu clase dezechilibrate. Frecventa: standard pentru clasificare.

### 5.3. K-Fold Cross-Validation

Logica: imparte datele in K parti, foloseste rotativ K-1 pentru train si 1 pentru test. Repeti K ori, mediezi metricile.

```
from sklearn.model_selection import KFold, cross_val_score
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf)
```

Frecventa: 90% pentru ML clasic.

### 5.4. TimeSeriesSplit

Logica: K folduri cronologice, expanding window. Standardul pentru time series.

```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
```

### 5.5. Walk-forward validation

Ferestre fixe de antrenare-testare, glisate in timp. Cand: forecast in business (antrenezi pe 12 luni, evaluezi pe luna urmatoare).

### 5.6. GroupKFold

Pastreaza grupurile (ex. pacientii) intregi in train sau test. Cand: multiple observatii per entitate. Frecventa: 100% in domenii medicale.

### 5.7. Nested Cross-Validation

CV in CV. Outer loop = evaluare model. Inner loop = tuning hyperparametri. Standardul de aur pentru estimari unbiased.

## 5.8. Conformal Prediction

Framework matematic care produce intervale de incredere statistically guaranteed. Cand: medical, financial, autonomous driving. Hot topic 2024-2025.

## Categoria 6. Modele de Machine Learning

### 6.1. Modele liniare

#### Linear Regression

Logica: gaseste coeficienti  $w$  care minimizeaza  $MSE(y_{true}, w^*X)$ . Solutie analitica:  $w = (X^T X)^{-1} X^T y$ . Cand: baseline, relatii liniare. Frecventa: 100% ca baseline.

#### Logistic Regression

Logica: regresie liniara peste care aplici functia sigmoid pentru a obtine probabilitate. Cand: clasificare binara, cand vrei probabilitati calibrate.

#### Ridge (L2)

Linear Reg + penalitate  $\lambda * ||w||^2$ . Cand: multicolaritate. Default in scikit-learn.

#### Lasso (L1)

Linear Reg + penalitate  $\lambda * ||w||_1$ . Feature selection automat (forteaza coef la 0).

#### Elastic Net

Combinatie L1 + L2. Cand: features corelate + selectie.

#### GLM (Generalized Linear Models)

Extinderi pentru target-uri non-gaussiene. Variante: Poisson (count data), Gamma (durate, costuri), Tweedie (insurance claims).

### 6.2. Modele bazate pe arbori

#### Decision Tree

Imparte spatiul features prin praguri. Pentru fiecare nod, alegi feature si prag care maximizeaza information gain (sau Gini). Avantaj: interpretabil. Limitari: overfitting sever.

#### Random Forest

N arbori antrenati pe sub-esantioane (bootstrap) cu sub-features aleatoare. Predictia = media (regresie) sau majority vote (clasificare). Avantaj: robust, fara nevoie de scalare. Frecventa: 80%, default pentru first try.

#### Extra Trees

Random Forest cu split-uri si mai aleatoare. Cand: cand RF e prea slow.

#### Gradient Boosting (concept)

Algoritm: 1) Antreneaza arborele 1 sa prezica  $y$ ; 2) Calculeaza reziduurile  $r_1 = y - \text{pred}_1$ ; 3) Antreneaza arborele 2 sa prezica  $r_1$ ; 4) Pred final =  $\text{pred}_1 + \text{pred}_2$ ; 5) Repeta.



## XGBoost

Gradient boosting cu regularizare avansata. Avantaje: regularizare L1+L2, parallel processing, handling missing values. Hyperparametri: n\_estimators, max\_depth, learning\_rate, subsample. Frecventa: #1 in Kaggle 2015-2023.

## LightGBM

Microsoft. Leaf-wise growth (vs level-wise XGBoost). Mai rapid pentru date mari. Avantaje: 5-10× mai rapid. Frecventa: in crestere, #1 in competitii recente.

## CatBoost

Yandex. Handling automat al features categorice. Avantaje: zero-friction cu features categorice. Cand: dataset-uri cu multe variabile categorice.

## 6.3. Retele neurale

### MLP (Multi-Layer Perceptron)

Retea feed-forward simpla. Fiecare strat =  $\text{activation}(W \cdot x + b)$ . Cand: date tabulare cand boosting nu e suficient. Limitari: necesita scaling.

### CNN

Filtre convolutionale care detecteaza pattern-uri locale. Cand: imagini, sunete, time series cu structura locala. Arhitecturi: ResNet, VGG, EfficientNet.

### RNN / LSTM / GRU

RNN: fiecare pas proceseaza input + hidden state. Limitari: vanishing gradient. LSTM: 3 porti (forget, input, output). GRU: simplificare LSTM (2 porti). LSTM in declin in NLP, inca standard in time series industrial.

### Transformers

Foloseste self-attention. Matematic:  $\text{Attention}(Q, K, V) = \frac{\text{softmax}(Q \cdot K^T / \sqrt{d})}{\text{softmax}(Q \cdot K^T / \sqrt{d})} \cdot V$ . Avantaje: paralelizabil, dependinte pe distante mari. Variante: BERT (encoder), GPT (decoder), T5. Dominanta absoluta in NLP.

## 6.4. Modele specializate time series

### ARIMA / SARIMA

AutoRegressive + Integrated (differencing) + Moving Average. Necesita serii stationare. Cand: time series simple. Frecventa: in declin.

### Prophet

Meta. Model aditiv:  $y = \text{trend} + \text{seasonality} + \text{holidays} + \text{noise}$ . Avantaje: interpretabil, robust la NaN si outliers. Frecventa: foarte popular pana 2022.

## N-BEATS / N-HiTS

Retele neurale pure pentru time series. Bazat pe blocuri de procesare. State-of-the-art in benchmark-uri academice.

## Temporal Fusion Transformer (TFT)

Combinatie LSTM + attention. Foarte interpretabil (attention visualization). Frecventa: in crestere puternica.

## Foundation models

TimeGPT (Nixtla), Chronos (Amazon), Lag-Llama. Pre-antrenate pe milioane de serii. Folosesti zero-shot. Frecventa: emergent foarte rapid 2024-2025.

## 6.5. AutoML

Librarii: AutoGluon, FLAML, auto-sklearn, H2O AutoML. Automatizare totala (FE + selection + model + tuning). Cand: baseline rapid.

## Categoria 7. Hyperparameter Tuning

### 7.1. Grid Search

Incearca exhaustiv toate combinatiile dintr-o grila. Limitari: explozie combinatorica. Cand: pentru spatii mici.

### 7.2. Random Search

Combinatii aleatoare. Avantaj: surprinzator de eficient. Bergstra & Bengio (2012) au aratat ca deseori bate Grid Search. Frecventa: standard pentru proiecte rapide.

### 7.3. Bayesian Optimization (Optuna)

Foloseste rezultatele anterioare pentru a alege inteligent urmatoarea combinatie. Construiesc un surrogate model (Gaussian Process sau Tree Parzen Estimator). Avantaje: 5-10× mai eficient decat Grid Search.

```
import optuna

def objective(trial):
    params = {
        'max_depth': trial.suggest_int('max_depth', 3, 10),
        'learning_rate': trial.suggest_float('lr', 0.01, 0.3, log=True),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
    }
    model = XGBRegressor(**params)
    cv_score = cross_val_score(model, X, y, cv=5).mean()
    return -cv_score

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=100)
```

Frecventa: Default in 2025. Inlocuieste Grid Search aproape complet.

### 7.4. Hyperband si BOHB

Hyperband: testeaza multe combinatii pe budget mic, elimina cele slabe, continua. BOHB: combinatia cu Bayesian. Cand: cand antrenarea e foarte lunga.

## Categoria 8. Metrici de Evaluare

### 8.1. Pentru regresie

#### MSE / RMSE

MSE =  $\text{mean}((y_{\text{true}} - y_{\text{pred}})^2)$ . RMSE =  $\sqrt{\text{MSE}}$ . Penalizeaza erorile mari. Cand: cand erorile mari sunt critice.

#### MAE

$\text{mean}(|y_{\text{true}} - y_{\text{pred}}|)$ . Robust la outliers. Cand: cand outliers reprezinta evenimente neimportante.

#### R<sup>2</sup>

$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$ . Cat din varianta target-ului explica modelul. Cand: pentru a compara intre modele. Limitari: poate fi negativ.

#### MAPE / sMAPE / MASE

MAPE: eroare procentuala. Cand: cross-domain. Limitari: simetric. sMAPE: simetric, standard in M-Competitions. MASE: raporteaza eroarea la un baseline naiv.

### 8.2. Pentru clasificare

#### Accuracy, Precision, Recall, F1

Accuracy:  $(\text{TP} + \text{TN}) / \text{Total}$ . Cand: clase echilibrate. Precision:  $\text{TP} / (\text{TP} + \text{FP})$ . Cand: FP costa mult. Recall:  $\text{TP} / (\text{TP} + \text{FN})$ . Cand: FN costa mult. F1: media armonica - vrei un singur numar.

#### AUC-ROC si AUC-PR

AUC-ROC: aria sub ROC curve. Cand: ranking. Limitari: optimist pentru clase dezechilibrate. AUC-PR: mai onest pentru imbalanced.

#### Confusion Matrix, Cohen's Kappa, MCC

Confusion Matrix: tabel TP/FP/FN/TN. Cohen's Kappa: accuracy ajustat pentru sansa. MCC: robust pentru clase dezechilibrate. Standardul 2025 pentru imbalanced binary classification.

### 8.3. Calibration

Brier Score: cat de bine calibrate sunt probabilitatile. ECE (Expected Calibration Error): diferenta intre probabilitatile prezise si frecventele reale. Cand: medical, financial.

## Categoria 9. Explainability (XAI)

### 9.1. Coefficients (linear models)

Interpretare directa. Cresterea cu 1 a feature X creste y cu  $w_X$ . Limitari: doar liniare.

### 9.2. Tree Feature Importance

Cat reduce impuritatea fiecare feature peste toti arborii. Limitari: bias spre features cu cardinalitate mare.

### 9.3. Permutation Importance

Logica: shuffle feature, vezi cat scade performanta. Avantaj: model-agnostic, robust.

### 9.4. SHAP (Shapley Additive Explanations)

Logica: foloseste teoria jocurilor (Lloyd Shapley, Nobel) pentru a calcula contributia echitabila a fiecarui feature.

Matematic:  $\phi_i = \sum_{S: i \in S} \frac{|S|! (n - |S|)!}{n!} (v(S \cup \{i\}) - v(S))$

Output: Global (importanta medie a fiecarui feature), Local (pentru o predictie individuala).

Vizualizari: Summary plot, Force plot, Dependence plot.

```
import shap
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X)
shap.summary_plot(shap_values, X)
shap.dependence_plot('feature_name', shap_values, X)
```

Frecventa: 100% in proiecte serioase. Apare in 90% din anunturile pentru ML Engineer.

### 9.5. LIME

Logica: in jurul unei predictii, antreneaza un model liniar simplu pe perturbari ale input-ului. Avantaj: model-agnostic. Limitari: explicatii pot fi instabile. Frecventa: in declin in favoarea SHAP.

### 9.6. PDP, ICE, ALE

PDP (Partial Dependence Plots): cum variaza predictia cand variezi un singur feature, mediind peste celelalte. ICE: pentru fiecare instanta separat. ALE: alternativa moderna la PDP, robusta la features corelate.

### 9.7. Counterfactual explanations

Logica: ce trebuie sa schimbi pentru a primi alta predictie? Util pentru utilizatori finali. Frecventa: in crestere - standardul pentru right to explanation GDPR.

### 9.8. Attention visualization si Integrated Gradients

Attention: pentru transformers - vezi pe ce parti din input se concentreaza modelul. Integrated Gradients: pentru DL - cuantifica contributia fiecarui input pixel/token.

# Categoria 10. Deployment

## 10.1. Model Serialization

pickle / joblib: standardul Python. ONNX: format universal, model exportat din orice framework. MLflow Model Format: include metadata, semnături I/O. TensorFlow SavedModel / PyTorch state\_dict: pentru deep learning.

## 10.2. API Serving

Flask: simplu, single-threaded. FastAPI: modern, async-native, validare automata. Default 2025. BentoML: framework specializat ML. TorchServe / TF Serving: serving pentru DL.

## 10.3. Containerization

Docker: imagini portabile. Standard 100%. Docker Compose: pentru aplicatii multi-container. Kubernetes: orchestrare la scala.

## 10.4. Streaming

Apache Kafka: pipeline evenimente. Apache Flink / Spark Streaming: procesare in streaming. Cand: predictii in timp real.

## Categoria 11. Monitoring

### 11.1. Data Drift

PSI (Population Stability Index): standard in banking.  $PSI > 0.2$  = drift semnificativ. KL Divergence: asimetric. Wasserstein Distance: mai robust decat KL. Evidently AI / WhyLabs: librarii Python complete.

### 11.2. Concept Drift

ADWIN, DDM, Page-Hinkley: algoritmi pentru detectie schimbari in stream-uri. Cand: stream learning, online ML.

### 11.3. A/B Testing

Comparatie statistica intre 2 versiuni de modele in productie. Cand: deploy nou model. Frecventa: 100% in companii mari.

### 11.4. Shadow Deployment

Noul model ruleaza in paralel cu vechiul, dar nu influenteaza deciziile reale. Cand: deploy cu risc redus.

## Categoria 12. Tehnici Emergente (2024-2025)

### 12.1. RAG (Retrieval-Augmented Generation)

Logica: LLM + vector database. Inainte sa raspunzi, cauti context relevant. Componente: 1) Embed documents → store in vector DB; 2) Embed query → retrieve top-K similar documents; 3) LLM raspunde cu context. Frecventa: explodare 2024-2025. Standard pentru chatbots cu knowledge.

### 12.2. LoRA / QLoRA

Fine-tuning eficient LLM-uri. Modifici doar ~1% din parametri (matrici low-rank). Avantaj: fine-tuning Llama-7B pe un GPU consumer. Frecventa: explozie.

### 12.3. Vector Databases

Pinecone, Weaviate, Chroma, Qdrant, Milvus. Cand: similarity search la scala (milioane embeddings). Frecventa: standard pentru RAG.

### 12.4. Function Calling / Tool Use

LLM-uri care pot apela functii externe (calculator, API, database). Frecventa: standard 2024+.

### 12.5. Multi-modal models

Inteleg simultan text + imagini + audio. Modele: GPT-4o, Gemini, Claude 3. Cand: aplicatii avansate.

### 12.6. Causal Machine Learning

Nu doar ce e corelat, ci ce cauzeaza ce. Librarii: DoWhy (Microsoft), CausalML, EconML. Cand: identificarea efectelor unei interventii.

### 12.7. Federated Learning

Antrenare distribuita fara a centraliza datele. Modelele locale trimit gradientele la server. Cand: privacy critic.

### 12.8. Differential Privacy

Garantie matematica ca outputul modelului nu releva date despre indivizi din training. Cand: cand antrenezi pe date sensibile.

### 12.9. Foundation models pentru time series

TimeGPT, Chronos, Lag-Llama, Moirai. Forecast cu zero-shot pe orice serie. Frecventa: explozie 2024-2025.



## Cum sa folosesti aceasta carte de referinta

Pentru un task nou: identifici categoria → vezi tehnicile → alegi 2-3 candidate → testezi.

Pentru a-ti pregati interviul: parcurgi de 2-3 ori. Memorezi top 5 din fiecare categorie.

Pentru a comunica profesionist: cand discuti un proiect, mentionezi tehnica + categorie + alternativele luate in considerare.

Acest document acopera 12 categorii mari si peste 150 de tehnici. Continutul reflecta state-of-the-art-ul Machine Learning din 2024-2025.